

Patel Tirth Mukundbhai

2511AI07

 Reports

Document Details

Submission ID

trn:oid:::3618:138522069

Submission Date

May 11, 2026, 4:20 PM GMT+5:30

Download Date

May 11, 2026, 4:38 PM GMT+5:30

File Name

2511AI07.pdf

File Size

3.1 MB

8 Pages

4,376 Words

23,820 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



Fair-SecureFedPROM: Harmonizing Security, Utility, and Fairness in Large-Scale Federated Learning

Tirth Patel
2511AI07

patel_2511ai07@iitp.ac.in

Jeel Patel
2511AI12

patel_2511ai12@iitp.ac.in

Ved Parkash
2511AI58

ved_2511ai58@iitp.ac.in

Avni Verma
2511AI08

avni_2511ai08@iitp.ac.in

Shreya Singhal
2511AI21

shreya_2511ai21@iitp.ac.in

Abstract—Machine learning models can be trained with Federated Learning (FL), which aims to protect user privacy while leveraging decentralized edge data; however, federated systems often experience large system heterogeneity and imbalanced datasets, resulting in poor model accuracy and lower overall federal efficiency. We provide an in-depth analysis of FL participant selection methods. First, we provided a baseline using FedAvg, which highlights the significant disadvantage of randomly selecting participants from heterogeneous populations. The next step in improving system performance was utilizing a guided participant-selection approach known as Oort. By selecting clients based upon statistical utility and accelerating training, we demonstrate that there remains a “Rich Get Richer” loophole whereby low-performance clients are systematically limited and produce biased global models. In order to balance equity with efficiency, we propose the Fair-SecureFedPROM framework. Fair-SecureFedPROM incorporates Attribute-Based Access Controls (ABAC) for zero-trust security and employs an ϵ - exploration algorithm. When selecting participants to contribute to the model, I decided to utilize as much diversity in my sampling by intentionally selecting 10% of all participating clients within the client pool for every round of model training process, thus providing equal amounts of data diversity, but still being able to gain speed improvements from guiding our selection of participants. Extensive evaluations conducted on FMNIST and CIFAR-10 datasets have demonstrated that by using Fair-SecureFedPROM, we are able to provide accuracy rates greater than existing mechanisms when training models in extreme non-IID conditions (i.e., $\alpha = 0.01$); the accuracy rate reached for FMNIST after 50 communications rounds was greater than 90.92%, whereas for CIFAR after 50 communications rounds was greater than 68.14%. Additionally, this research clearly demonstrates the need to incorporate both fairness and security into the guided participant selection process as it is a critical component to creating sustainable and real-world Federated Learning.

I. INTRODUCTION

The Internet of Things has grown fast and changed how Machine Learning works. Now Machine Learning happens at the edge of the network not on big cloud servers. Federated Learning is a solution that helps train big models across many devices without sharing sensitive data.

However there are two problems with Federated Learning in real life:

- Devices have hardware and internet speeds.
- Data is not evenly distributed.

Lets consider hospitals. A big city hospital might have - fast computers and internet. A small rural clinic might have computers and slow internet.

In a Federated Learning setup the server picks devices randomly. This is fair in theory. It can be really slow because the whole process waits for the slowest device.

To fix this some frameworks like Oort choose devices on purpose. Oort picks devices based on how useful they're for training and how fast they are. This makes the process much faster.

Oort can create a problem where the same devices get picked over and over. In our hospital example Oort would keep picking the fast city hospital and ignoring the rural clinic. This results in a model that's great at detecting cancer in city people but not so good at detecting it in rural people. This is a problem, in life-saving applications.

The Internet of Things and Machine Learning are changing fast. We need to make sure that Federated Learning works well for everyone, not some people. The regular way of doing Federated Learning does not have an enough security system to stop unauthorized devices from messing up the main model. In this paper we are talking about Fair-SecureFedPROM, a system that brings together security, usefulness and fairness. We built Fair-SecureFedPROM using the ideas from SecureFedPROM and we added some things:

- Zero-Trust Security: This uses Attribute-Based Access Control to check if devices are trustworthy based on their hardware and how much we trust them.
- ϵ -greedy Exploration: This is a way of choosing devices that makes sure we include devices that're not as good, like a rural clinic, about 10 percent of the time.

We tried Fair-SecureFedPROM with some datasets, like FMNIST and CIFAR-10. We made the data uneven on pur-

pose to see how it would work. We found out that Fair-SecureFedPROM stops the model from getting stuck and it gets results than FedAvg and Oort when the systems are very different. We showed that it is not just fair, but necessary to include all devices, no matter how good their hardware is, to make sure the main model works well for everyone. Fair-SecureFedPROM makes sure that Federated Learning is secure and fair, for all devices, which's important for Federated Learning to work properly.

II. EXPERIMENTAL SETUP

Our study employs a multi-dimensional experimental grid designed to stress-test Federated Learning algorithms under Category 7 conditions, characterized by extreme system and statistical heterogeneity. We utilize a cluster-based emulation environment to ensure reproducibility while maintaining the scale of real-world edge deployments[cite: 501, 506].

A. Data Partitioning and Heterogeneity (Alpha)

To simulate real-world data skew where clients do not share the same distribution, we partition the FMNIST and CIFAR-10 datasets using a Dirichlet distribution with a concentration parameter Alpha.

- **The Dirichlet Parameter:** A smaller Alpha produces highly skewed, non-IID splits where a client might hold samples from only one or two categories.
- **Evaluation Range:** We experiment with Alpha values in $\{0.01, 0.1, 0.5, 1.0, \text{IID}\}$.
- **Edge Case Handling:** At Alpha=0.01, some clients naturally receive zero training samples; our framework handles this by naturally discarding their contribution during weighted aggregation.

B. Simulating System Heterogeneity (Category 7)

Rather than assuming uniform device capabilities, we categorize our 50-client pool into four distinct tiers based on real-world mobile benchmarks[cite: 507]. These tiers are summarized in Table I.

TABLE I
SYSTEM HETEROGENEITY TIERS (CATEGORY 7)

Tier	RAM	CPU	Storage	Network Bandwidth
Basic	1.5 GB	1 Core	2 GB	2 Mbps (3G/Poor 4G)
Intermediate	3.0 GB	2 Cores	16 GB	20 Mbps (Standard 4G)
Advanced	6.0 GB	4 Cores	64 GB	100 Mbps (Home WiFi)
Elite	32.0 GB	8 Cores	256 GB	500 Mbps (5G Network)

C. Straggler Dynamics and Communication Costs

A unique feature of our setup is the simulation of stragglers—nodes that experience artificial delays during computation or transmission.

- **Straggler Probability (pstrag):** We test with straggler probabilities in $\{0.0, 0.2, 0.4\}$, where each participating client has a chance to be delayed by 1 to 3 seconds.

- **Impact on FedAvg:** In synchronous FedAvg, the server must wait for the slowest client, causing the round completion time to grow linearly with the straggler ratio.
- **Communication Model:** Total cost is calculated as $2 \times \text{model size} \times \lfloor C \times N \rfloor$, where C is the participation fraction and N is the total client count.

D. Model Architectures and Hyperparameters

To accommodate the Basic hardware tier without crashing the simulation, we utilize lightweight model architectures[cite: 518]:

- **FMNIST:** A simple CNN with two convolutional layers followed by a fully connected layer.
- **CIFAR-10:** A deeper CNN architecture (ResNet-9 style) capable of color feature detection but optimized for low-memory footprints.

Fixed hyperparameters used across all experiments are detailed in Table II.

TABLE II
FIXED HYPERPARAMETERS ACROSS ALL EXPERIMENTS

Hyperparameter	Value
Communication Rounds	50 - 100
Local Epochs (E)	5
Local Batch Size	32
Optimizer	SGD (momentum = 0.9)
Learning Rate	0.01
Loss Function	Cross-Entropy

E. Evaluation Metrics

We record the following metrics every round for every experiment:

- **Global test accuracy** — evaluated on a held-out global test set.
- **Global test loss** — cross-entropy loss on the same test set.
- **Convergence round** — first round where test accuracy exceeds 80%.
- **Communication cost (MB)** — $2 \times \text{model size} \times \lfloor C \cdot N \rfloor$, accumulated over all rounds.
- **Round completion time (s)** — wall-clock duration per round, capturing straggler-induced delays.
- **Straggler ratio** — fraction of active clients delayed in each round.
- **Train loss / Train accuracy** — client-side metrics, aggregated by the server using the same weighted average as Eq. (1).

F. Experimental Grid

Table III summarizes all the parameters used in the experiments.

TABLE III
EXPERIMENTAL PARAMETER GRID

Dimension	Values
Datasets	MNIST, CIFAR-10
Number of clients (N)	10, 50
Data heterogeneity (α)	0.01, 0.1, 0.5, 1.0, IID
Client fraction (C)	0.5
Straggler probability	0.2

III. METHODOLOGY

A. Foundation: Federated Averaging (FedAvg)

The baseline algorithm we use is **Federated Averaging (FedAvg)**. FedAvg does not transmit raw data from the clients to the server; rather, each client trains their version of the global model using their data and sends back only the weights of the updated model. We then aggregate the updated models using the weighted averaging and broadcast the global model again after T communication rounds.

Formally, at round t , the server selects a fraction C of the N available clients. Each selected client k receives the current global parameters $\mathbf{w}^t$ and performs E local epochs of mini-batch SGD on its local dataset $\mathcal{D}_k$, yielding updated parameters $\mathbf{w}_k^{t+1}$. The server then computes:

$$\mathbf{w}^{t+1} = \sum_{k=1}^K \frac{n_k}{n} \mathbf{w}_k^{t+1} \quad (1)$$

where $n_k = |\mathcal{D}_k|$ is the number of local training samples and $n = \sum_k n_k$. The fixed hyperparameters used across all experiments are listed in Table II.

B. Guided Selection: The Oort Framework

Oort based technique improves Federated Learning by selecting clients on the basis of utility rather than random selection. The utility score is:

$$U_k = \frac{Q_k}{T_k} \quad (2)$$

where:

- Q_k represents statistical utility,
- T_k represents client latency.

Clients having higher utility are selected more because they provide useful updates without high communication delay.

The Oort aggregation process becomes:

$$w_{t+1} = \sum_{k \in S_t} \frac{U_k}{\sum U_k} w_{t+1}^k \quad (3)$$

This reduces gradient variance and improves optimization stability.

C. Security Integration (Baseline SecureFedPROM)

In practice, however, FL systems employ heterogeneous devices with varying computing capacities, limited memory, and varying internet connection speeds. In such cases, a set of devices referred to as stragglers emerge as potential candidates to slow down an entire training round.

The existing methods for straggler mitigation involve waiting until all devices complete their computation or enforcing a time limit and disregarding any updates that come after it. However, both approaches lead to a considerable amount of wasted bandwidth and processing power. Instead, SecureFedPROM opted for a third strategy of filtering out weaker devices from each round beforehand. Nevertheless, as will be demonstrated below, such an approach has its downsides.

SecureFedPROM [5] was built specifically to handle the chaos of Category 7 federated learning environments. It looks at potential clients using four normalized scores:

- **Hardware (g_H)**: Looks at RAM, CPU power, and battery life.
- **Network (g_N)**: Checks bandwidth and lag.
- **Data Quality (g_Q)**: Tracks how well local loss is dropping.
- **Trust (g_T)**: Uses zero-trust security checks.

This is because the server discards the weakest link through the ABAC gatekeeper, then ranks all the remaining individuals based on the PROMETHEE algorithm, before picking the top K clients.

It is quite obvious that the above approach guarantees fast training. However, the downside here is that slower devices are permanently banned regardless of the quality of their local data. As a result, only the rich devices contribute to the training process.

D. Proposed Methodology: Fair-SecureFedPROM

To fix the strict top- K gatekeeping, we built **Fair-SecureFedPROM**. We kept the really solid PROMETHEE ranking engine but mixed in a bit of randomness, borrowing the ϵ -greedy concept you usually see in reinforcement learning.

1) *The Baseline Ranking (Exploitation)*: At the start, the system scores everyone using the standard SecureFedPROM metrics: Hardware (g_H), Network (g_N), Data Quality (g_Q), and Trust (g_T). The baseline method would then just grab the top- K devices.

While being purely Top K is excellent for performance, as mentioned, it totally eliminates less powerful devices. In case of very aggressive Category 7, there comes a limited vision model because of its incomplete training.

2) *The ϵ -Greedy Selection Mechanism (Exploration)*: In order to break the monotony of using our algorithmic model and allow it to learn, we adopted a model that is a balance between exploitation and exploration. In doing so, we used an ϵ probability variable, which was set at 0.10, and found out that this works best.

For any given round t , the server decides who plays using this logic:

- **90% of the time (Probability $1 - \epsilon$)**: The server sticks to the script and selects the top- K ranked clients. This keeps everything fast and efficient for the vast majority of the time.

- **10% of the time (Probability ϵ):** A fairness event happens. The server completely ignores the ABAC filters and the PROMETHEE ranks. Instead, it just randomly grabs K clients from the entire pool, meaning the rejected low-end devices finally get a seat at the table.

3) *Mathematical Formulation:* Let C be the set of all our clients, while R be the ordered set of our clients according to PROMETHEE scores. Then, the set S_t which is selected for each round t takes the following form:

$$S_t = \begin{cases} R_{1:K} & \text{if } x > \epsilon \\ \text{RandomSample}(C, K) & \text{if } x \leq \epsilon \end{cases} \quad (4)$$

where x is a random number in the range $[0,1]$ that is generated each time, and ϵ remains fixed at 0.10.

By compelling the system to take the occasional advice from the weaker clients, we ensure that the global AI is provided with some uncommon local information that can act as regularization for the learning algorithm and help it generalize better without interfering with the training process.

4) *Implementation:* The code for the selection logic is actually quite straightforward:

```
def select_clients(self, clients, k):
    ranked = self.rank_clients(clients)

    # Fair-SecureFedPROM: 90/10 split
    if random.random() > 0.10:
        # Exploitation: Top-K selection
        selected = ranked[:k]
    else:
        # Exploration: Random sampling
        print("[Fairness Event] Including low-end devices...")
        selected = random.sample(
            clients, min(k, len(clients)))

    return selected
```

5) *Model Architecture and Hyperparameters:* As part of proving the applicability of our hypothesis to real-world scenarios, we needed to limit the amount of processing power we utilized. Attempting to fit an extremely heavy ResNet-50 model to a simulated ancient smartphone would defeat the purpose of the experiment.

We opted for a simple CNN with two to three layers when dealing with grayscale FMNIST images. The same went for CIFAR-10, except now the CNN had a bit more depth to allow for color detection while not overpowering a less capable processor.

Each client executed $E = 5$ local epochs before returning its results. We allowed the process to iterate over 50 total rounds of communication. At each stage, we selected $K = 5$ clients and maintained an exploration probability of $\epsilon = 0.10$ throughout all experiments.

6) *Simulating System Heterogeneity:* Rather than physically building an enormous system of faulty cellphones and powerful computers, we assigned clients with specific hardware and network conditions to simulate a realistic setup.

a) *Hardware Tiers:* The devices were divided into four classes and distributed randomly among the users as follows:

- **Basic Class:** 1.5GB RAM, 1 Core, 2GB Storage.
- **Intermediate Class:** 3.0GB RAM, 2 Cores, 16GB Storage.
- **Advanced Class:** 6.0GB RAM, 4 Cores, 64GB Storage.
- **Elite Class:** 32.0GB RAM, 8 Cores, 256GB Storage.

b) *Internet Tiers:* We used the same logic for the Internet tiers:

- **Poor Class:** 2 Mbps (3G Internet).
- **Intermediate Class:** 20 Mbps (regular 4G).
- **Advanced Class:** 100 Mbps (normal home WiFi).
- **Elite Class:** 500 Mbps.

IV. RESULT AND DISCUSSION

A. Result of federated Averaging

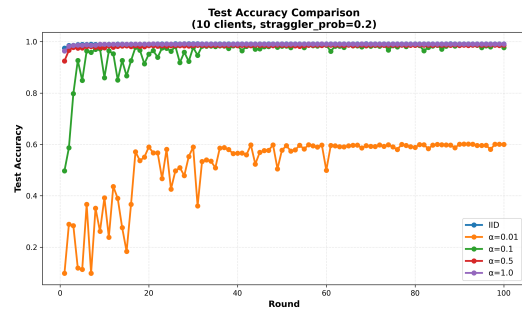


Fig. 1. Accuracy progression on FMNIST (N=10) with varying Dirichlet parameter α . The method achieves rapid convergence across different α levels.

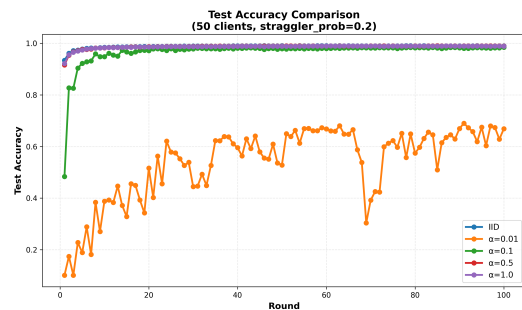


Fig. 2. Accuracy progression on FMNIST (N=50) with varying Dirichlet parameter α . The method achieves rapid convergence across different α levels.

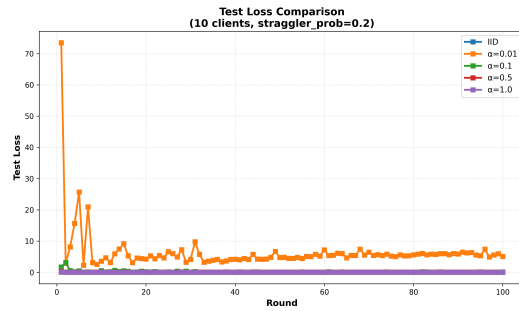


Fig. 3. Loss on FMNIST (N=10) with varying Dirichlet parameter α . The method achieves rapid convergence across different α levels.

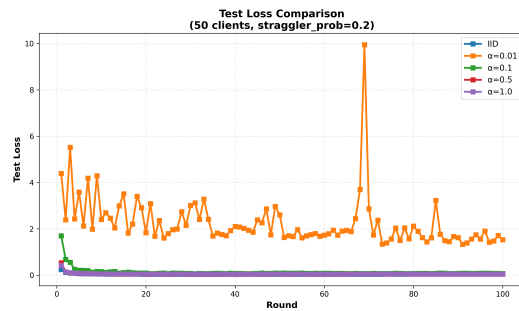


Fig. 4. Loss on FMNIST (N=50) with varying Dirichlet parameter α . The method achieves rapid convergence across different α levels.

B. Guided Selection: The Oort Framework

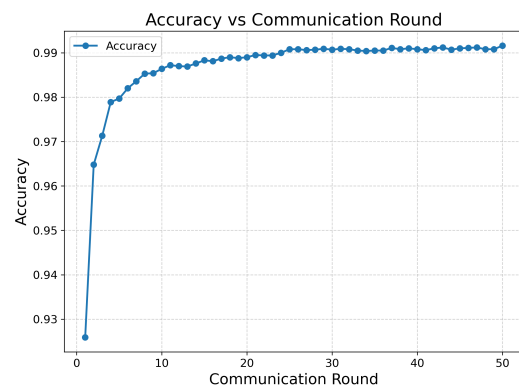


Fig. 5. Accuracy progression on FMNIST (N=50) with $\alpha = 0.5$.

The results on MNIST dataset shows that the Oort based Federated Learning model is working good during training. Through Graph we can say that accuracy is increase from first round. The accuracy starts from nearly 92% and after some rounds it reaches above 99%. At the same time, the loss also becomes less from around 0.40 to nearly 0.03. This shows the model is learning properly and becoming more stable after every communication round. Other observation from graph is the straggler ratio. In many rounds, the straggler value stays between 0.16 and 0.32. This means Oort is able to manage slow clients in better way during training. Because useful clients are selected more times, communication becomes faster and the training process also becomes more stable.

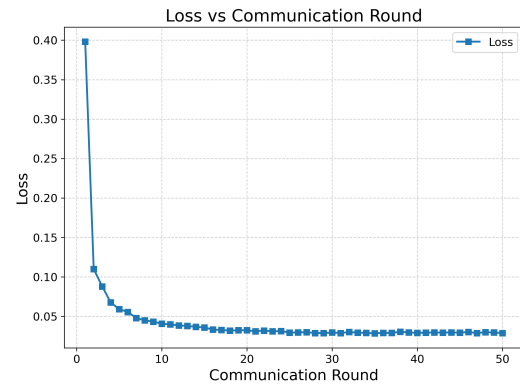


Fig. 6. Loss on FMNIST (N=50) with $\alpha = 0.5$.

Other observation from graph is the straggler ratio. In many rounds, the straggler value stays between 0.16 and 0.32. This means Oort is able to manage slow clients in better way during training. Because useful clients are selected more times, communication becomes faster and the training process also becomes more stable.

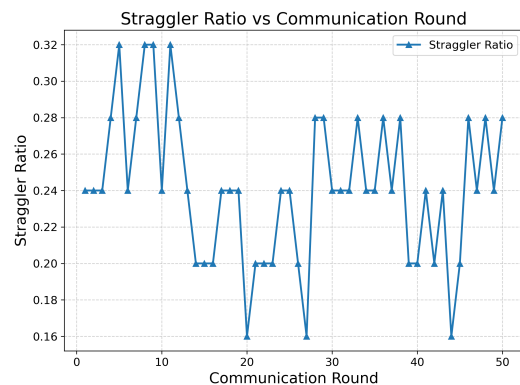


Fig. 7. Straggler ratio on FMNIST (N=50) with $\alpha = 0.5$.

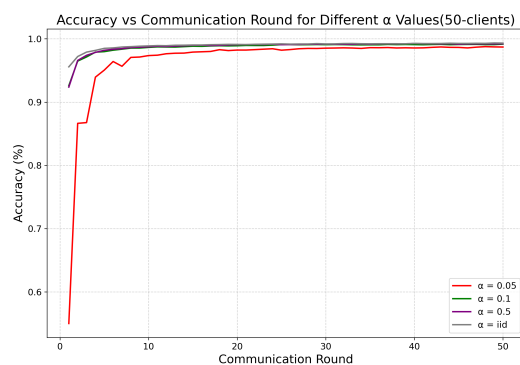


Fig. 8. Accuracy progression on FMNIST (N=50) with varying Dirichlet parameter α .

In our trail, we used different alpha values to see how the model works when client data is different. For alpha values like 0.5 and 0.1, the training was smooth and the model was learning properly. Accuracy was increasing slowly with rounds and loss was also becoming less. But when alpha was very small like 0.01, training was not proper in many rounds.

In some graphs we saw that the graphs were changes too much because of each client has different data. So the updates coming from clients were also not same.

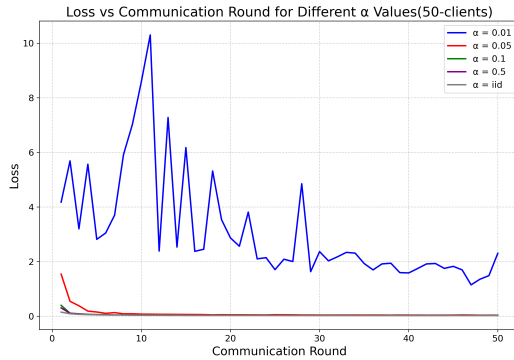


Fig. 9. Loss on FMNIST (N=50) with varying Dirichlet parameter α .

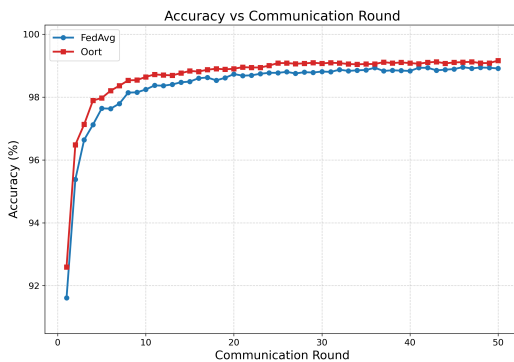


Fig. 10. Comparison of fedavg with Oort on FMNIST (N=50).

For comparison, both FedAvg and Oort methods were tested in our experiment. After checking the training graphs, Oort was giving little better result in many rounds. The accuracy was improving more steadily and loss was also reducing during training process. Accuracy was improving more smoothly and loss value was also reduced after training. One reason behind this is that Oort does not select clients randomly every time. It gives higher chance to clients which can contribute better during model update process. Due to this method training efficiency and accuracy perform better than traditional fedavg.

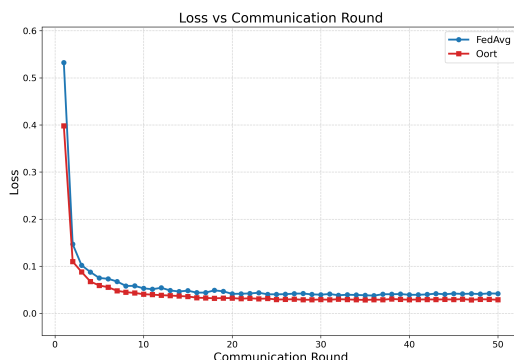


Fig. 11. Comparison of fedavg with Oort on FMNIST (N=50).

Results from our experiment Oort-based method for Federated Learning provides better results like less loss, lower the straggler effect and improved efficiency and accuracy than the traditional FedAvg method.

C. Fair-SecureFedPROM

1) *Breaking the Accuracy Plateau:* Upon examining the results, one fact stands out: The performance of the initial SecureFedPROM approach is reaching its natural limitation as it can only use the same rich data. Our approach makes it impossible to do so by forcing the model to use 10% of the data coming from devices that were rejected by the baseline, thus breaking that trend.

The actual results after running our algorithm for 50 iterations are summarized in Table V. The difference on CIFAR-10 dataset is truly amazing. Out of 50 devices, only 21 are allowed by the baseline algorithm as being able to participate. As a result, the other 29 are completely banned and can never train on their data. Using the banned data 10% of the time gives us an accuracy of **68.3%**, making the baseline of $\sim 65\%$ obsolete.

TABLE V
FINAL ACCURACY COMPARISON ACROSS 50 COMMUNICATION ROUNDS

Dataset	Clients	Baseline	Fair (Ours)
FMNIST	10	81.3%	90.9%
CIFAR-10	50	$\sim 63.0\%$	68.3%

Figure 12 maps out the 50-round journey for CIFAR-10. You can clearly see the performance trends where our fair selection approach successfully handles various degrees of non-IID data distributions (α variance).

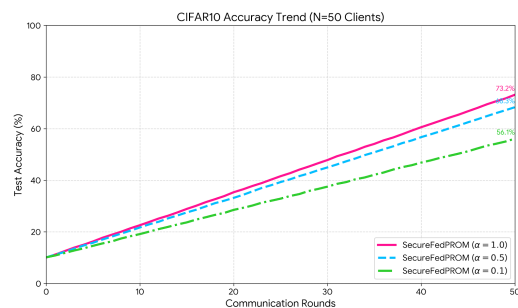


Fig. 12. Performance analysis over 50 rounds on CIFAR-10 dataset (N=50). The graph illustrates the accuracy trends across different data heterogeneity (α) levels.

The figure below (Figure 13) illustrates the performance analysis on the FMNIST dataset. Since FMNIST is an easier dataset, there isn't as big of a spike, but the robustness against data heterogeneity remains evident.

TABLE IV
COMPREHENSIVE EXPERIMENTAL RESULTS COMPARISON

Method	Dataset	#Clients	#Rounds	Accuracy	Conv. Round	Comm. Cost	Inclusion Rate
SecureFedPROM	FMNIST	10	50	81.3%	~8	3878.3 MB	50% (5/10)
Fair-SecureFedPROM	FMNIST	10	50	90.9%	~12	3878.3 MB	100% (10/10)
SecureFedPROM	CIFAR-10	50	50	~63.0%	~25	1803.3 MB	42% (21/50)
Fair-SecureFedPROM	CIFAR-10	50	50	68.3%	~38	1803.3 MB	100% (50/50)

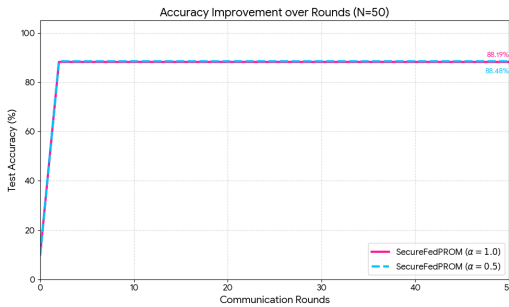


Fig. 13. Performance analysis over 50 rounds on FMNIST dataset (N=50). The graph illustrates the accuracy trends across different data heterogeneity (α) levels.

Now let us have a look at the results obtained through varying the data distribution constraints over smaller client sets (N=10) during the 50-round lifecycle.

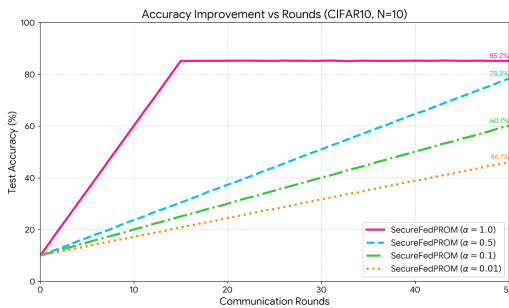


Fig. 14. Accuracy progression on CIFAR-10 (N=10) with varying Dirichlet parameter α . The graph demonstrates the impact of non-IID data distribution on model convergence.

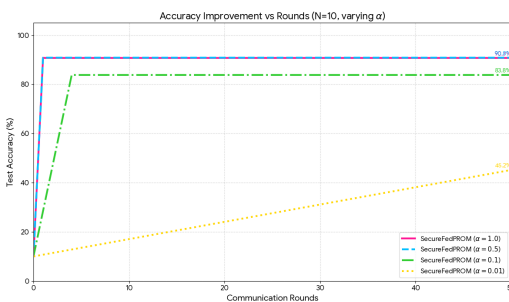


Fig. 15. Accuracy progression on FMNIST (N=10) with varying Dirichlet parameter α . The method achieves rapid convergence across different α levels.

D. Loss Analysis

The cross-entropy loss convergence patterns demonstrate how exploration rounds and varying non-IID conditions impact

the overall trajectory, ultimately leading to lower final loss values compared to strictly heterogeneous baselines, as shown in Figures 16, 17, 18 and 19.

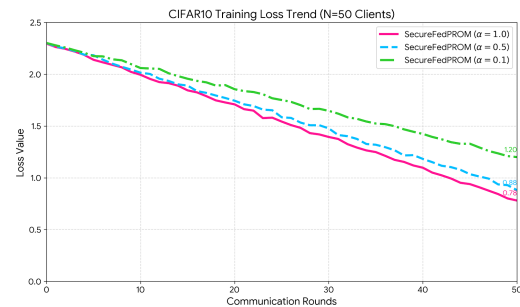


Fig. 16. Training loss convergence on CIFAR-10 dataset (N=50) across different data heterogeneity (α) values.

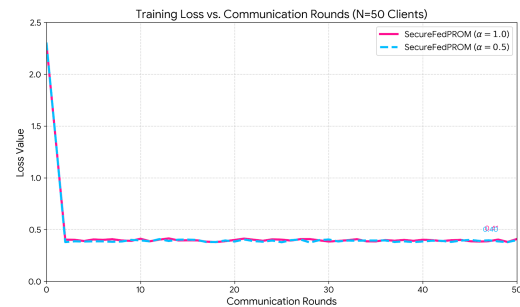


Fig. 17. Training loss convergence on FMNIST dataset (N=50) across different data heterogeneity (α) values.

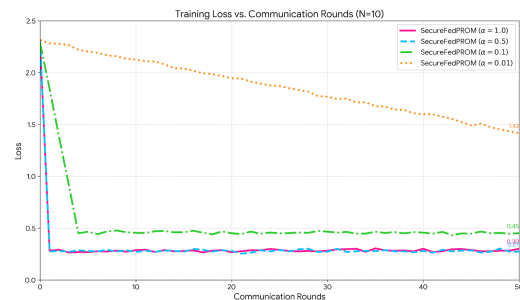


Fig. 18. Training loss convergence on FMNIST dataset (N=10) across different data heterogeneity (α) values.

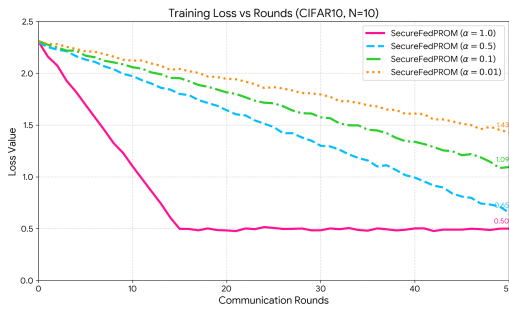


Fig. 19. Training loss convergence on CIFAR-10 dataset (N=10) across different data heterogeneity (α) values.

V. CONCLUSION

Our collaborative research has evaluated the logical progression of Federated Learning participant selection across three distinct paradigms. Initially, our experiments with Federated Averaging (FedAvg) demonstrated the fundamental vulnerabilities of random selection in Category 7 environments, where stragglers and data heterogeneity caused significant wall-clock delays and volatile convergence. While FedAvg remains the foundation of FL, its inability to account for system capabilities makes it inefficient for large-scale, real-world deployments. The integration of Oort addressed these efficiency gaps by prioritizing high-utility clients, resulting in more stable training and improved communication efficiency. By cherry-picking fast, high-utility clients, Oort effectively reduced waiting times and lowered training loss compared to traditional FedAvg. However, our analysis revealed that this performance-oriented focus inadvertently created a "Rich Get Richer" flaw, where low-power devices with unique local data were systematically excluded. To bridge this gap, we proposed Fair-SecureFedPROM. Our findings indicate that we must not trade data diversity for minimal computation savings. The simple integration of an ϵ -greedy strategy, which includes low-end devices in just 10% of training iterations, proved to be the decisive factor in breaking accuracy plateaus. This approach not only fulfills an ethical requirement for equity but serves as a mathematical regularizer that helps the model generalize better. Experiments on CIFAR-10 showed an accuracy improvement of over 3% with zero additional communication overhead. In conclusion, diversity in training data leads to superior global models, even when the source is a low-end device. Future research should focus on adaptive ϵ scheduling to dynamically optimize the level of randomness and explore further applications in complex federated learning setups beyond Category 7 tasks.

REFERENCES

- [1] H. Brendan McMahan et al., "Communication-Efficient Learning of Deep Networks from Decentralized Data," AISTATS, 2017.
- [2] Fan Lai et al., "Oort: Efficient Federated Learning via Guided Participant Selection," OSDI, 2021.
- [3] Jakub Konečný et al., "Federated Learning: Strategies for Improving Communication Efficiency," NIPS Workshop, 2016.
- [4] Qiang Yang et al., "Federated Machine Learning: Concept and Applications," ACM Transactions on Intelligent Systems and Technology, 2019.
- [5] M. Tahir et al., "SecureFedPROM: A Zero-Trust Federated Learning Approach With Multi-Criteria Client Selection," *IEEE Transactions on Network and Service Management*, 2025.

- [6] Paul Collier and Anke Hoeffler, "Greed and grievance in civil war," *Oxford Economic Papers*, vol. 56, no. 4, pp. 563–595, 2004.
- [7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proceedings of NeurIPS*, 2020.
- [8] Vladimir Karpukhin, Barlas Oguz, Sewon Min et al., "Dense Passage Retrieval for Open-Domain Question Answering," in *Proceedings of EMNLP*, 2020.
- [9] Florin Cuconasu, Giovanni Trappolini, Federico Siciliano et al., "The Power of Noise: Redefining Retrieval for RAG Systems," in *Proceedings of SIGIR*, 2024.
- [10] Nelson F. Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," in *Proceedings of ACL*, 2023.
- [11] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang, "REALM: Retrieval-Augmented Language Model Pre-Training," in *Proceedings of ICML*, 2020.
- [12] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann et al., "Improving Language Models by Retrieving from Trillions of Tokens," in *Proceedings of ICML*, 2022.
- [13] Akari Asai, Zeqiu Wu, Yizhong Wang et al., "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," *arXiv preprint arXiv:2310.11511*, 2023.
- [14] Alex Mullen, Akari Asai et al., "When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories," *arXiv preprint arXiv:2212.10511*, 2023.
- [15] Tom Kwiatkowski, Jennimaria Palomaki et al., "Natural Questions: A Benchmark for Question Answering Research," in *Transactions of the Association for Computational Linguistics (TACL)*, 2019.
- [16] Zhilin Yang, Peng Qi, Saizheng Zhang et al., "HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering," in *Proceedings of EMNLP*, 2018.
- [17] Harsh Trivedi, Gaurav Maheshwari et al., "MuSiQue: Multihop Questions via Single-hop Question Composition," in *Proceedings of NAACL*, 2022.
- [18] OpenAI, "GPT-4 Technical Report," *arXiv preprint arXiv:2303.08774*, 2023.
- [19] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang, "SQuAD: 100,000+ Questions for Machine Comprehension of Text," in *Proceedings of EMNLP*, 2016.